Chapter 4 – Inheritance

(*Object-Oriented Programming*)

Outline

4.1 Introduction

4.2 Base Class and Derived Class

4.3 Protected Members

4.4 Access Control in Derived Classes

4.5 Types of Inheritance

4.6 Constructors and Destructors in Derived Classes

4.7 Method Overriding in Derived Classes

4.8 Multiple Inheritance

4.1 Introduction

- What problem do you observe in the Teacher and Student classes?

Teacher
- id: string - name: string - coeff_salary: double
+ getID(): string + getName(): string + displayProfile(): void + getCoeff(): double + calSalary(): double

Student
- id: string - name: string - gpa: double
+ getID(): string + getName(): string + displayProfile(): void + getGPA(): double + rank(): string

The Problem: Code Duplication

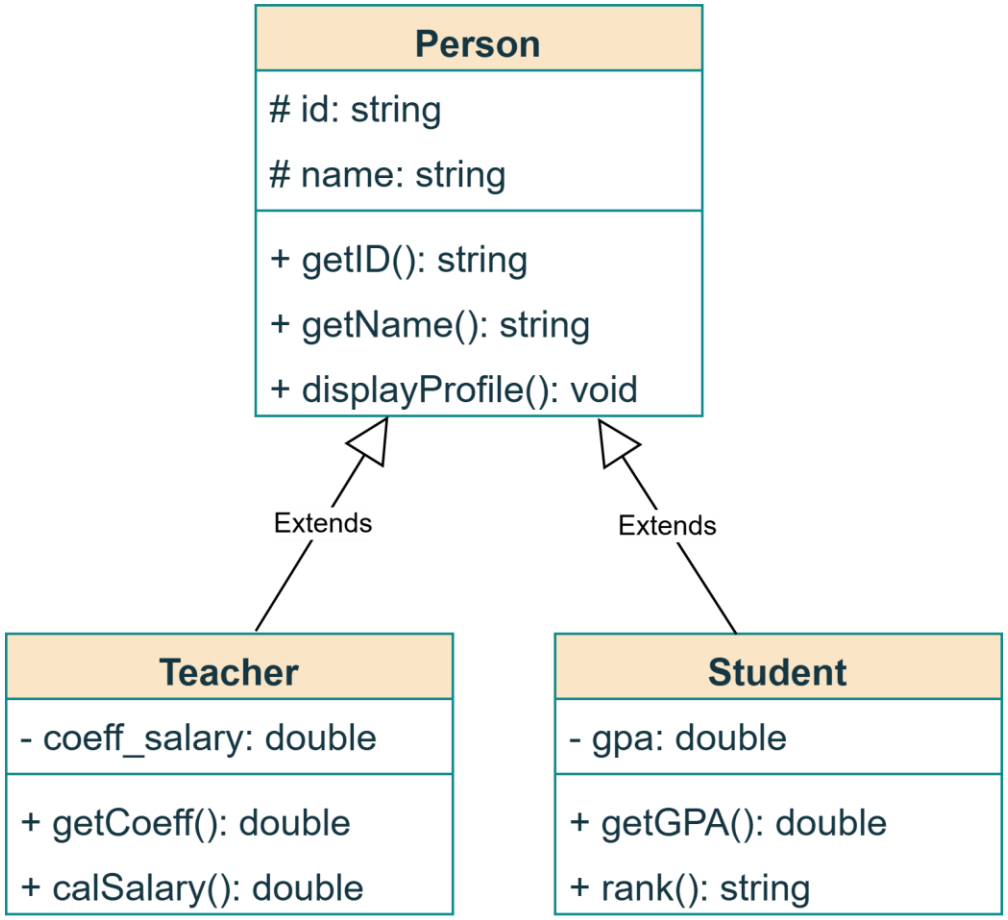
- **Observation**
  - Classes Teacher and Student share common attributes: id, name
  - They also share common behaviors: getID(), getName(), displayProfile()
- **Problem**
  - Code duplication across classes.
  - In a large system with 10+ classes, this becomes **unmanageable** → Difficult to maintain and extend.
- **Key Idea**
  - Can we define a **general class** that contains shared features?
  - And let other classes **reuse** those features?

Principle: DRY

- Don't Repeat Yourself (DRY)
  - Write shared logic once in the superclass.
  - All subclasses automatically inherit it.
  - Fix a bug in one place → fixed everywhere.

"The DRY principle is one of the most fundamental principles in software engineering. " – Hunt & Thomas, The Pragmatic Programmer

Better approach



Introduction to Inheritance

- **Definition**
  - Inheritance is a mechanism to create a new class from an existing class.
  - The new class (derived class) inherits attributes and methods from the base class.
  - The new class can also add new features or override existing ones.
- **Why use inheritance?**
  - Use existing functionality (reusability) and adapt it to new requirements.
  - Focus only on additional features instead of rewriting everything.
  - More robust and reliable (reused and tested code).

Inheritance in Real Life

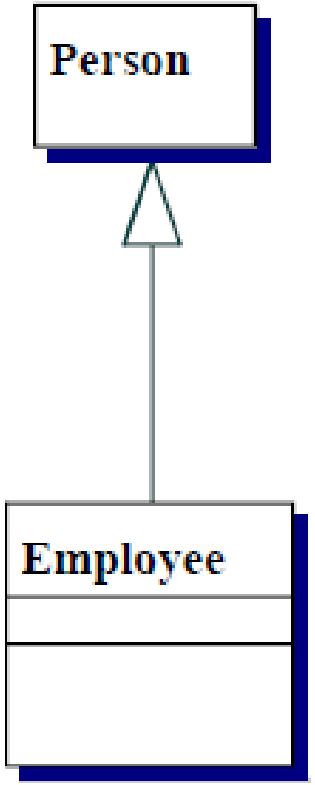
- People inherit traits from **parents and ancestors**
- Polygon → Triangle, Quadrilateral, Pentagon, Hexagon.
  - All share: vertices, edges
- Motor Vehicle → Car, Motorbike, Truck
  - All share: engine, movement capability
- Computer → Desktop, Laptop, Tablet
  - All share: CPU, memory, input/output devices

Advantages of Inheritance

- Saves time, cost, and development effort, avoids creation of objects from scratch.
- Improves program reliability because existing classes are often tested and reused in many applications.
- Avoids redundancy and maintaining consistency.
- Mapping a real-world hierarchy
  - Models real-world relationships naturally.
  - Makes programs more structured, readable, and extensible.

4.2 Base Class and Derived Class

- The relationship between a Base Class and a Derived Class is "is-a".
- Base classes tend to be more general and derived classes tend to be more specific.
- Every object of a Derived Class is an object of its Base Class.
- A Derived Class inherits all attributes and some methods from the Base Class, except:
  - Constructors, destructors, friend functions
- A Derived Class can **redefine (override)** inherited methods.

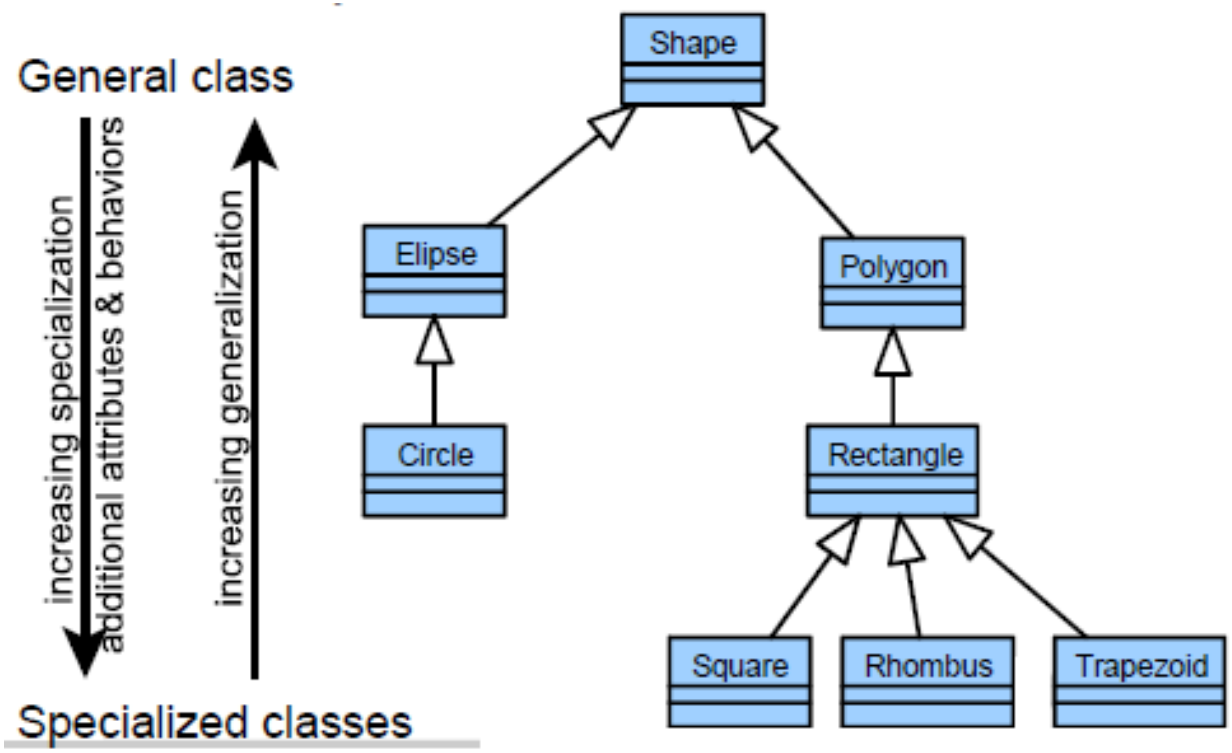


Base Class
Superclass
Parent Class
Generalization
Ancestor

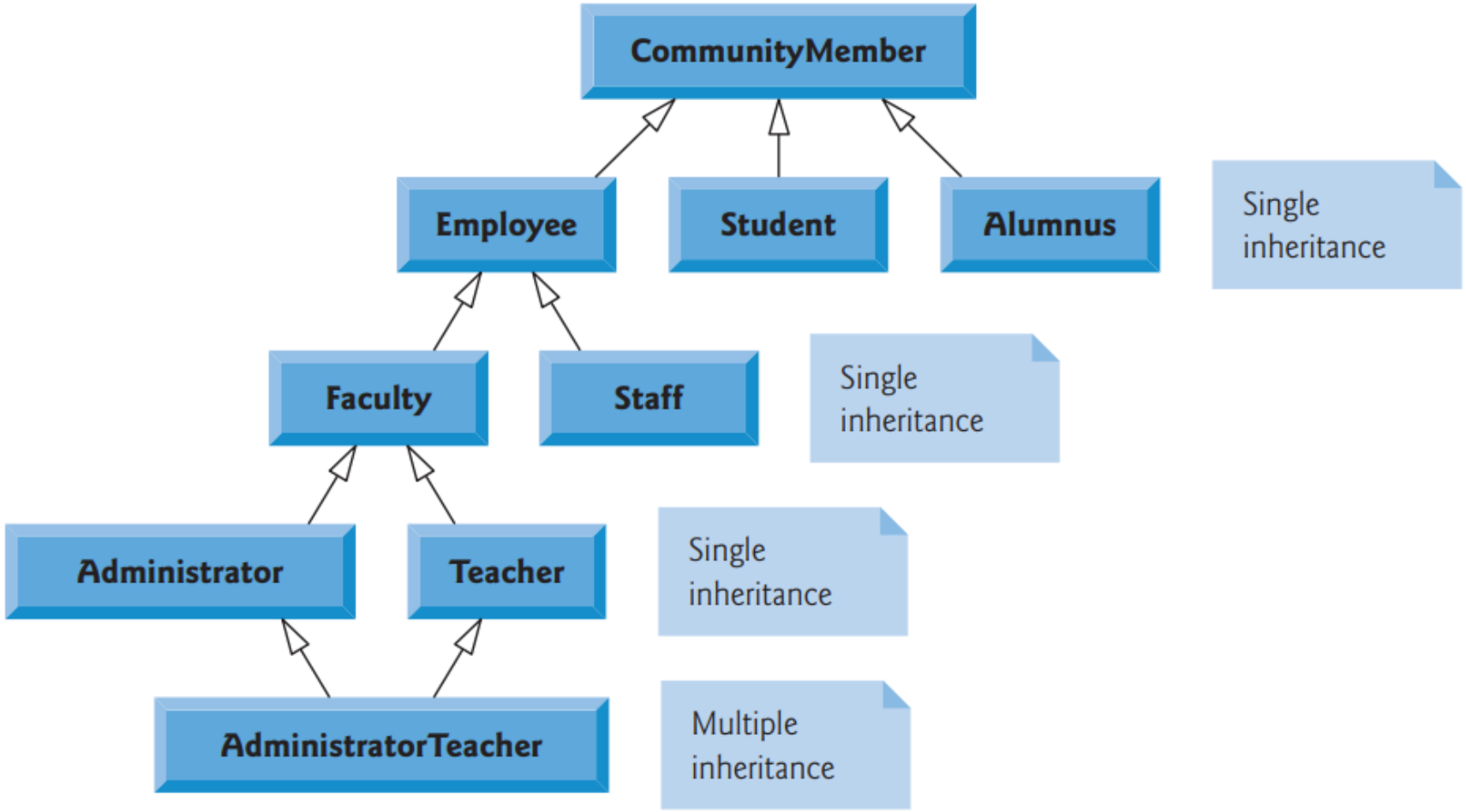
Derived Class
Subclass
Child Class
Specialization
Descendant

Class hierarchies

- Inheritance relationships form class hierarchies



Class hierarchies



Defining a Derived Class

- Syntax

```
class <DerivedClass> : <access_specifier> BaseClass {
    // additional members
};
```

- Access Specifier
  - public
  - private (default)
  - protected

Example

```
//Person.h

#pragma once
#include <iostream>
#include <string>
using namespace std;

class Person
{
private:
    string id;
    string name;
public:
    Person(string _id = "", string _name = "") : id(_id), name(_name) //constructor
    {}
    string getID()   const { return id; }
    string getName() const { return name; }
    friend ostream& operator<<(ostream& out, const Person&);
};
```

```
ostream& operator<<(ostream& out, const Person& p)
{
    out << "ID: " << p.id << endl;
    out << "Name: " << p.name << endl;
    return out;
}
```

Example

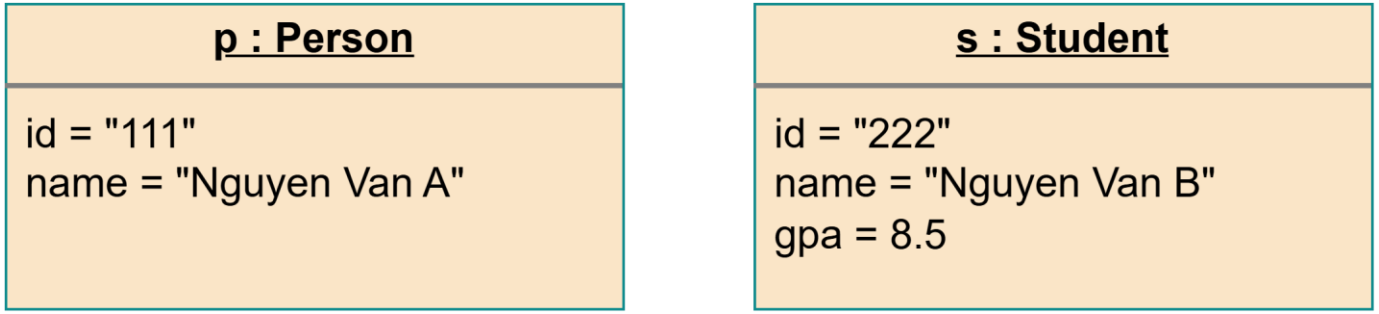
```
//Student.h
#pragma once
#include "Person.h"
class Student : public Person
{
private:
    double gpa;
public:
    Student(string _id = "", string _name = "", double _gpa = 0)
        : Person(_id, _name)
    {
        if (_gpa < 0 || _gpa > 10)  gpa = 0;
        else  gpa = _gpa;
    }
    double getGPA() const { return gpa; }
    string rank() const {
        if (gpa >= 9.0) return "Excellent";
        if (gpa >= 8.0) return "Very Good";
        if (gpa >= 7.0) return "Good";
        if (gpa >= 5.0) return "Average";
        return "Fail";
    }
};
```

Example

```
//main.cpp
#include "Person.h"
#include "Student.h"

int main()
{
    Person p("111", "Nguyen Van A");
    cout << p << endl;
    Student s("222", "Nguyen Van B", 8.5);
    cout << s << endl; //Call the << operator of the Person class

    return 0;
}
```

Example

- The memory is allocated for the two objects p and s.



4.3 Protected Members

- A base class's private members are:
  - accessible only within its body and to the friends of that base class.
  - not directly accessible from within the child class functions.
- Problem:
  - Child classes sometimes need access to the base class's private members.
  - But want to block outside access

→ C++ provides an intermediate access level: protected

- A base class's protected members can be accessed:
  - within the body of that base class, by members and friends of that base class
  - by members and friends of any classes derived from that base class

Example

```
//Student.h
#pragma once
#include "Person.h"
class Student : public Person
{
private:
    double gpa;
public:
     Student(string _id = "", string _name = "", double _gpa = 0)
        : Person(_id, _name)  {
        if (_gpa < 0 || _gpa > 10)  gpa = 0;
        else  gpa = _gpa;
    }
    double getGPA() const { return gpa; }
    string rank() const {
        if (gpa >= 9.0) return "Excellent";
        if (gpa >= 8.0) return "Very Good";
        if (gpa >= 7.0) return "Good";
        if (gpa >= 5.0) return "Average";
        return "Fail";
    }
    friend ostream& operator<<(ostream& out, const Student&);
};
```

```
ostream& operator<<(ostream& out, const Student& s)
{
    out << "ID: " << s.id << endl;
    out << "Name: " << s.name << endl;
    out << "GPA: " << s.gpa << endl;
    return out;
}
```

Error: cannot access private member declared in class 'Person'

Example

```
//Person.h
#pragma once
#include <iostream>
#include <string>
using namespace std;

class Person
{
protected:
    string id;
    string name;
public:
    Person(string _id = "", string _name = "") : id(_id), name(_name) //constructor
    {}
    string getID()   const { return id; }
    string getName() const { return name; }
    friend ostream& operator<<(ostream& out, const Person&);
};
```

Protected Members

- **Advantages:**
  - Derived classes can directly access member data of the base class
  - Reduces the need for getters/setters
- **Disadvantages:**
  - Derived classes may unintentionally modify base class data incorrectly
  - Creates dependency on the implementation of the base class

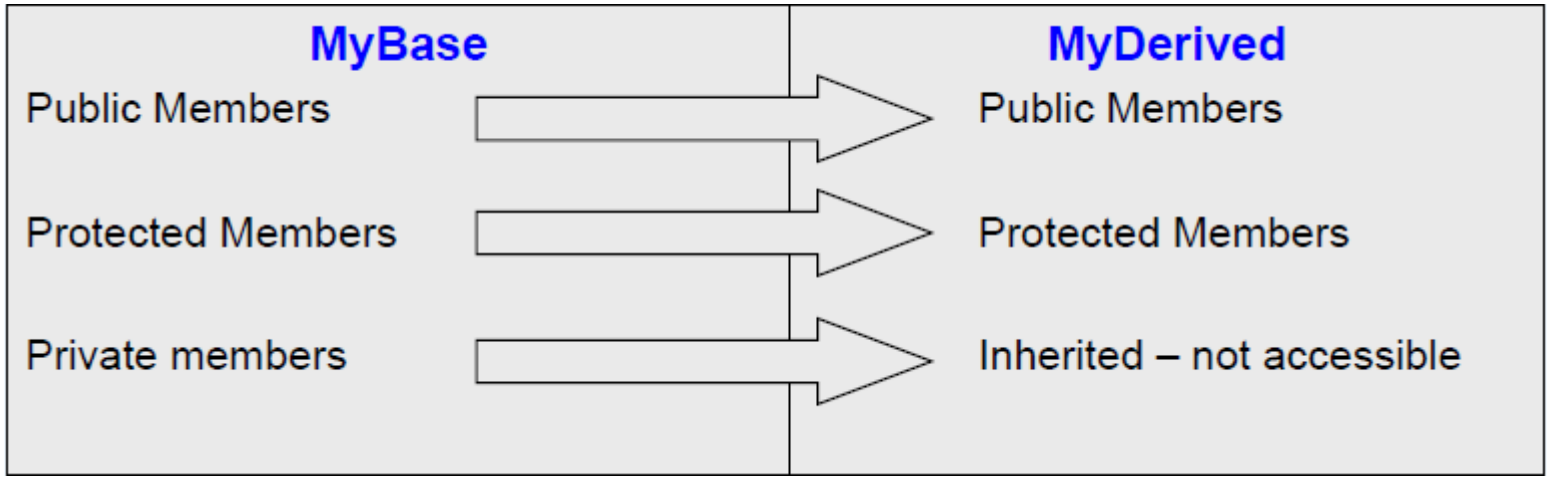
4.4 Access Control in Derived Classes

- When defining a derived class, three access specifiers – public, protected, and private – can be used before the base class name.

```
class Student : public Person
class Student : protected Person
class Student : private Person
```

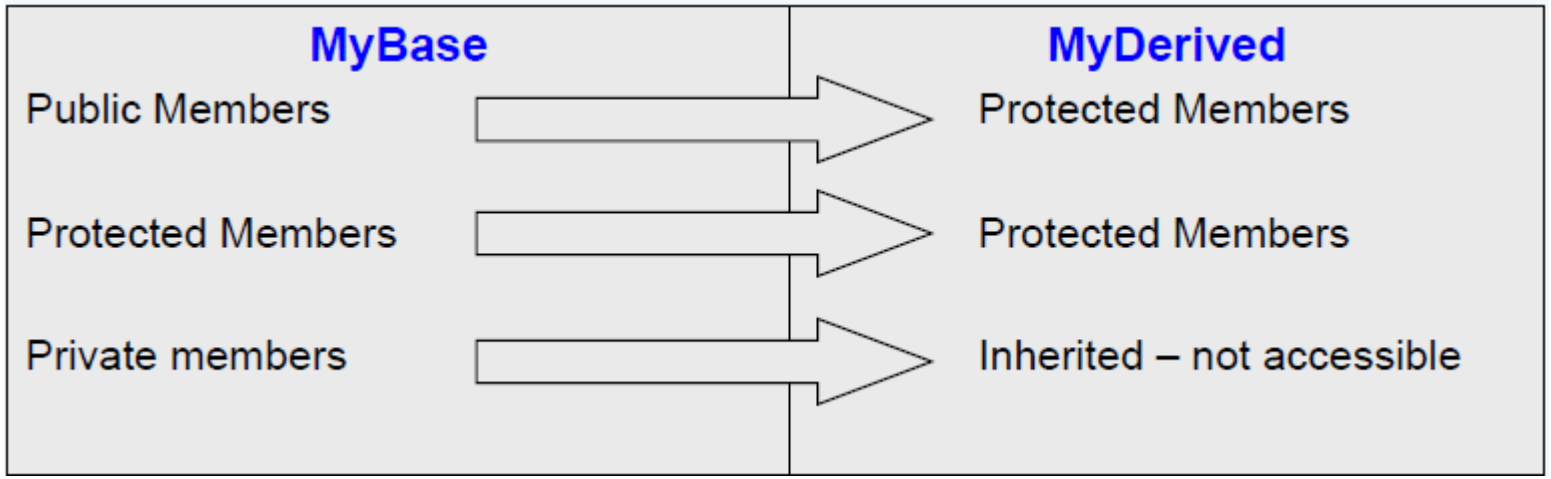
Public Inheritance

```
class MyDerived : public  MyBase{

};
```



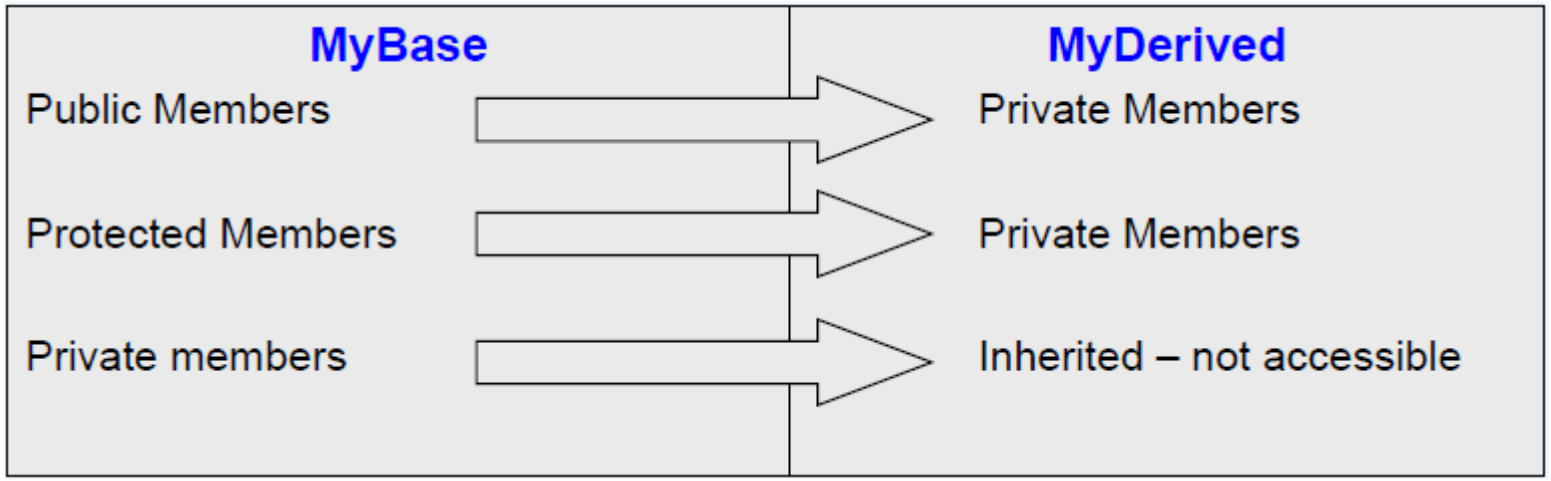
Protected Inheritance

```
class MyDerived : protected  MyBase{

};
```



Private Inheritance

```
class MyDerived : private  MyBase{

};
```



Access Control in Derived Classes

Type of Inheritance	Accessible inside Derived Class? i.e. In any of the methods of derived class			Accessible outside Derived Class? i.e. through derived class objects		
	Private member of Base class	Protected member of Base Class	Public member of Base class	Private member of Base class	Protected member of Base class	Public member of Base class
Private	**NO**	**YES**	**YES**	**NO**	**NO**	**NO**
Protected	**NO**	**YES**	**YES**	**NO**	**NO**	**NO**
Public	**NO**	**YES**	**YES**	**NO**	**NO**	**YES**

Access Control in Derived Classes

- Most commonly used: public inheritance
  - Does not change access levels of base class members
- Redeclare access levels of inherited members in in derived classes:

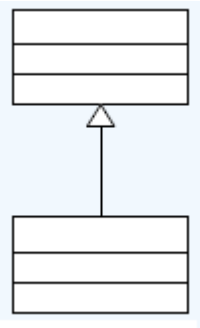
```
access_specifier:
    BaseClass::member;
```

  - Allows changing access of: public members and protected members
  - Provides flexibility in design
  - Enables increasing accessibility (e.g., protected → public) or restricting access when needed
  - Not strictly tied to inheritance type

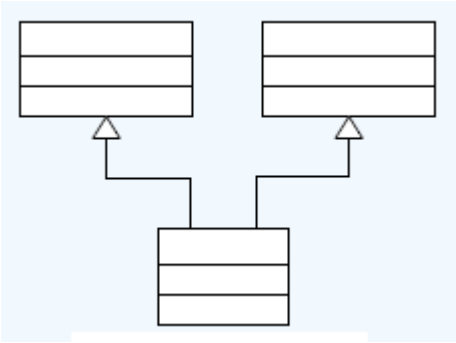
Access Control in Derived Classes

```
class Student : protected Person
{
public:
    Person::getID; //Redeclare access levels
    ...
};
```

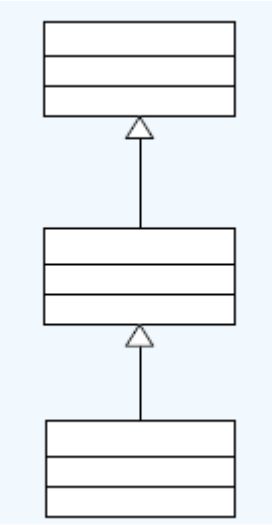
4.5 Types of Inheritance



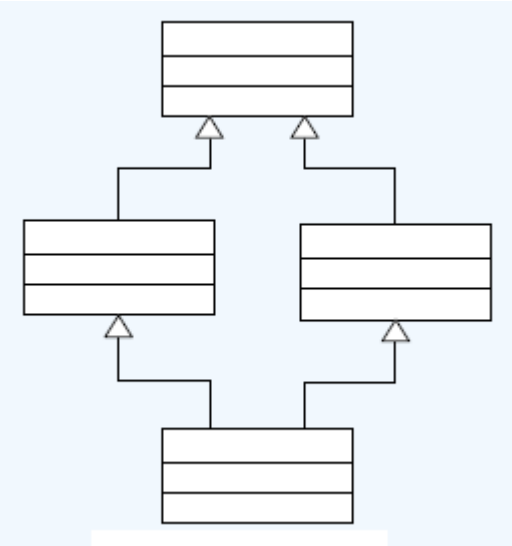
Single Inheritance



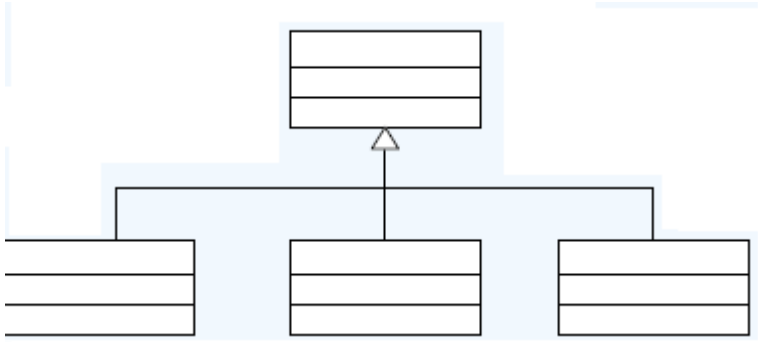
Multiple Inheritance



Multi-level Inheritance



Hybrid Inheritance



Hierarchical Inheritance

4.6 Constructors and Destructors in Derived Classes

- Constructors are not inherited but are invoked in derived classes.
- Creating a derived object triggers a chain of constructor calls
  - Base class data is initialized first
  - Derived class works on a fully initialized object
- The derived class constructor:
  - Calls its base class constructor first
  - Then executes its own logic
- Base constructors are called in two ways:
  - Explicit call (initializer list)
  - Implicit call (default constructor)

Constructors and Destructors in Derived Classes

- Explicit call to a base class constructor:

```
DerivedClass(parameters)
  : BaseClass(arguments)  // call the base class constructor
{
    //body
}
```

- Order of base class constructor calls in multi-level inheritance
  - From the top-most base class → down to derived class
- Order of base class constructor calls in multiple inheritance
  - In the order they appear in the inheritance list

Constructors and Destructors in Derived Classes

- When a derived-class object is destroyed, the program calls that object's destructor. This begins a chain (or cascade) of destructor calls.
- The derived-class destructor and the destructors of the direct and indirect base classes and the classes' members execute in reverse of the order in which the constructors executed
- Process:
  - Derived class destructor executes
  - Calls base class destructor
  - Continues up the hierarchy
  - Final step: Top-level base class destructor executes→ Object is removed from memory

Example

```
class Person
{
protected:
    string id;
    string name;
public:
    Person(string _id = "", string _name = "") : id(_id), name(_name)
    {
        cout << "Person's constructor is called" << endl;
    }
    ~Person() { cout << "Person's destructor is called" << endl; }
    //...
};
```

Example

```
Person's constructor is called
ID: 111
Name: Nguyen Van A

Person's constructor is called
Student's constructor is called
ID: 222
Name: Nguyen Van B
GPA: 8.5

Student's destructor is called
Person's destructor is called
Person's destructor is called
```

```
class Student : public Person
{
private:
    double gpa;
public:
    Student(string _id = "", string _name = "", double _gpa = 0)
        : Person(_id, _name)
    {
        cout << "Student's constructor is called" << endl;
        if (_gpa < 0 || _gpa > 10)  gpa = 0;
        else  gpa = _gpa;
    }
    ~Student() { cout << "Student's destructor is called" << endl; }
    //...
};
```

4.7 Method Overriding in Derived Classes

- **Problem**:
  - Inherited methods may not always fit the derived class
  - Base class provides a general implementation
  - Derived classes may require specialized behavior
- **Solution**: Method Overriding
  - Allows us to redefine a member function of the class inherited from the base class.
  - The name and function signature, i.e., parameters and data types, are the same in the base and derived classes
  - The behaviour is altered to suit the specific needs of the derived class.

Example

```
class Person
{
protected:
    string id;
    string name;
public:
    Person(string _id = "", string _name = "") : id(_id), name(_name)
    {
        cout << "Person's constructor is called" << endl;
    }
    ~Person() { cout << "Person's destructor is called" << endl; }
    string getID()   const { return id; }
    string getName() const { return name; }
    void displayProfile() const {
        cout << "ID: " << id << endl;
        cout << "Name: " << name << endl;
    }
};
```

Example

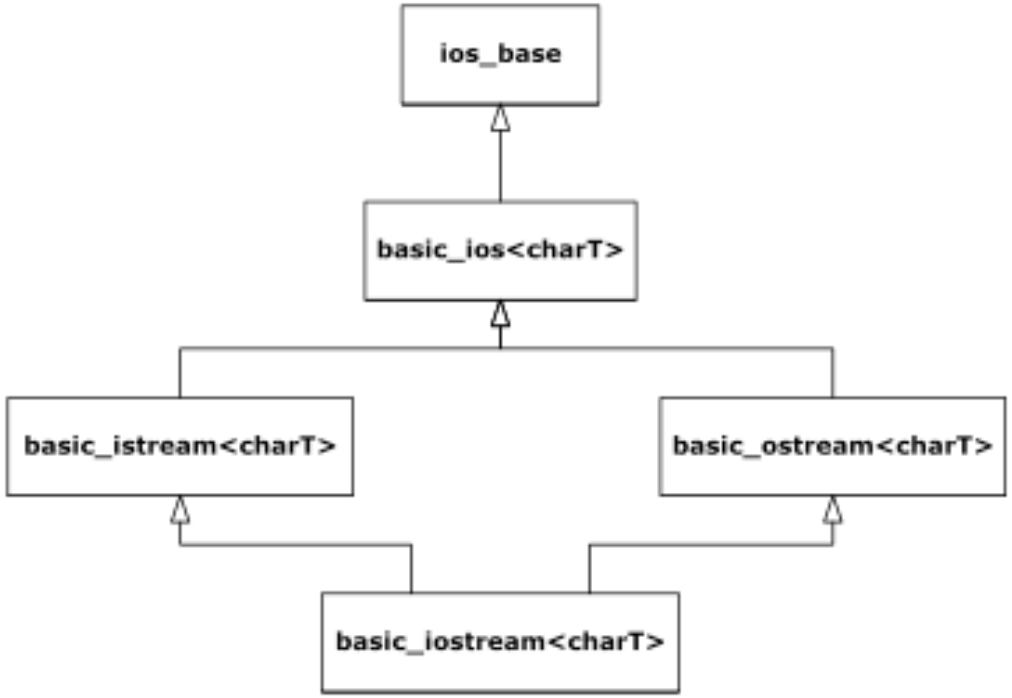
```
class Student : public Person
{
private:
    double gpa;
public:
    Student(string _id = "", string _name = "", double _gpa = 0)
        : Person(_id, _name)
    {
        cout << "Student's constructor is called" << endl;
        if (_gpa < 0 || _gpa > 10)  gpa = 0;
        else  gpa = _gpa;
    }
    ~Student() { cout << "Student's destructor is called" << endl; }
    void displayProfile() const { //method overriding
         // Call the displayProfile() function of the Person class
        Person::displayProfile();
        cout << "GPA : " << gpa << endl;
    }
    //...
};
```

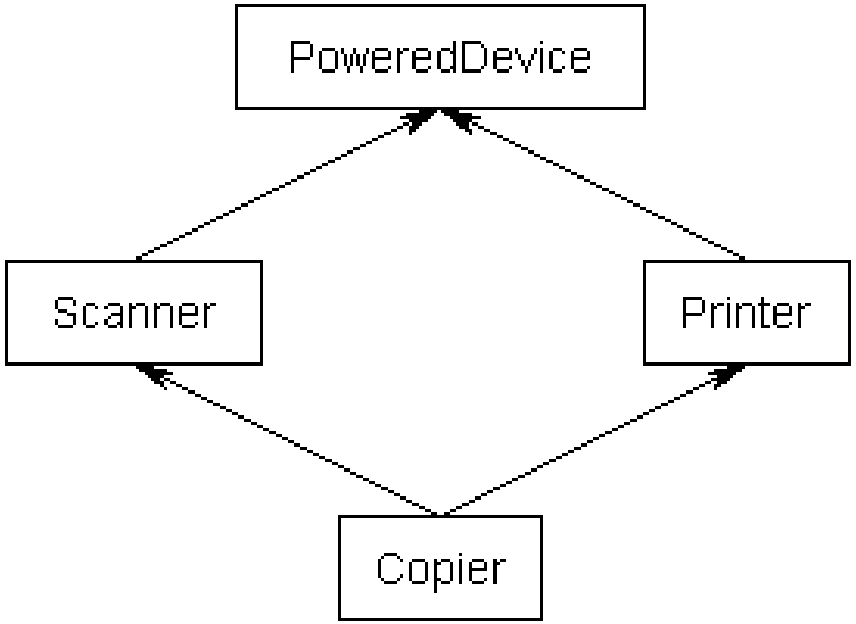
Example

```
//main.cpp
#include "Person.h"
#include "Student.h"

int main()
{
    Person p("111", "Nguyen Van A");
    // Call the displayProfile() function of the Person class
    p.displayProfile();

    Student s("222", "Nguyen Van B", 8.5);
    // Call the displayProfile() function of the Student class
    s.displayProfile();

    return 0;
}
```

4.8 Multiple Inheritance

- Multiple inheritance allows a class to inherit from more than one base class
- Purpose:
  - Reuse attributes and methods from multiple sources
  - Combine functionalities into a single class





Multiple Inheritance

- Syntax

```
class Derived : public Base1, public Base2, ... {
    ...
};
```

- Example:
  - Create a class Date
  - Create a class Time
  - Create a class DateTime using multiple inheritance
    - Combine both date and time
    - Provide a method to display full date and time

Example

```
//Date.h
#pragma once
#include <iostream>
using namespace std;

class Date {
protected:
    int day, month, year;
    bool isLeapYear() const;
    int lastDayOfMonth() const;

public:
    Date(int d = 1, int m = 1, int y = 1900);
    bool isValidDate(int d, int m, int y) const;
    void setDate(int d, int m, int y);
    void displayDate() const;
};
```

Example

```
//Time.h
#pragma once
class Time {
protected:
    int hour, minute, second;
public:
    Time(int h = 0, int m = 0, int s = 0);
    bool isValidTime(int h, int m, int s) const;
    void setTime(int h, int m, int s);
    void displayTime() const;
};
```

Example

```
//DateTime.h
#pragma once
#include "Date.h"
#include "Time.h"

class DateTime : public Date, public Time {
public:
    DateTime(int d, int m, int y, int h, int min, int s)
        : Date(d, m, y), Time(h, min, s) {
    }

    void displayDateTime() const {
        displayDate();
        cout << " ";
        displayTime();
        cout << endl;
    }
};
```

Example

```cpp
//main.cpp
#include "DateTime.h"

int main() {
    DateTime dt1(31, 2, 2023, 25, 70, 80); // invalid → default
    dt1.displayDateTime();

    DateTime dt2(4, 4, 2026, 21, 30, 0); // valid
    dt2.displayDateTime();

    return 0;
}
```

Problems in Multiple Inheritance

- Multiple inheritance is a powerful design mechanism.
- However, it introduces several issues.
  - Member Ambiguity
    - Same member names may exist in multiple base classes.
      → Leads to ambiguity during access.
  - Diamond problem
    - A class may inherit the same base class multiple times.

Member Ambiguity

```
//Student.h

#pragma once
class Student
{
protected:
    string id;
public:
    Student(void);
    string getID() const;
    ~Student(void);
};
```

```
//Employee.h

#pragma once
class Employee
{
protected:
    string id;
public:
    Employee(void);
    string getID() const;
    ~Employee(void);
};
```

Member Ambiguity

```
#pragma once
#include "Student.h"
#include "Employee.h"

class TeachingAssistant : public Student, public Employee {
public:
    TeachingAssistant(void): Student(), Employee() {}

    ~TeachingAssistant(void) {}

    void changeID(string _id) {
        id = _id;
    }
};
```

error C2385: ambiguous access of 'id'
could be the 'id' in base 'Student'
or could be the 'id' in base 'Employee'

Member Ambiguity

```
#include <iostream>
#include "TeachingAssistant.h"
using namespace std;

int main() {
    TeachingAssistant* t = new TeachingAssistant();
    cout << t->getID();
    delete t;
    return 0;
}
```

error C2385: ambiguous access of 'getID‘ could be the 'getID' in base 'Student‘ or could be the 'getID' in base 'Employee'

Member Ambiguity

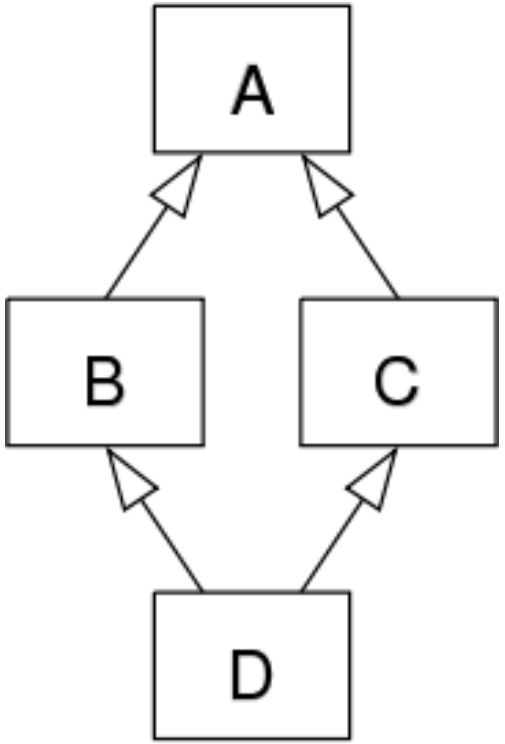
- Solution:
  - Use scope resolution operator (::) to specify the base class

```
void TeachingAssistant::changeID(string _id){
    Employee::id = _id;  // specify Employee's id
}

int main()
{
    TeachingAssistant *t = new TeachingAssistant();
    cout << t->Employee::getID();  // call Employee's method
    delete t;
    return 0;
}
```

Diamond problem

- Class D inherits from class A twice.

```
class A {};
class B : public A {};
class C : public A {};
class D : public B, public C {};
```



Diamond problem

```
class Person {
protected:
    string id;
public:
    Person(string _id = "") : id(_id) {}
    string getID() const { return id; }
};

class Employee : public Person
{...};
class Student: public Person
{...};
class TeachingAssistant: public Student, public Employee
{...};
```

The TeachingAssistant class has two versions of id and getID(), inherited separately from Student and Employee.

Diamond problem

- Solution: Virtual inheritance
- The keyword virtual should precede the derivation to provide a virtual base class, that is, a class with only one instance inherited when inherited from multiple paths.

```
class Student : virtual public Person
{...};

class Employee : virtual public Person
{...};

class TeachingAssistant : public Student, public Employee
{...};
```

  - Only one shared instance of Person
  - Eliminates duplication of id, getID()
  - Removes ambiguity when accessing members

Diamond problem

- Important Note
  - The **most derived class** must initialize the virtual base class.
- The virtual keyword is used when Student and Employee inherit from Person, the construction of Student and Employee does not prompt the construction of Person.
- TeachingAssistant must construct all of its ancestors.

```
TeachingAssistant(string id)
    : Person(id), Student(), Employee()
{}
```